SIMATS ENGINEERING

SAVEETHA SCHOOL OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

OBJECT ORIENTED PROGRAMMING WITH C++

ASSIGNMENT 1 – CO1

DESIGN, IMPLEMENT AND VALIDATE THE FARE LOGIC
OF A CITY METRO SMART-CARD SYSTEM

Submitted by

Student Name : R. Peeran Basha

Register Number: 192472244

Section / Batch: Slot-B

DECLARATION AND SUBMISSION NOTE

This report is prepared from the questions and requirements in the uploaded Object Oriented Programming with C++ Assignment 1 (CO1). The source assignment requires individual execution, a GitHub repository, progressive commits, source files, input files and output screenshots. Fill in the register number, submission date and working GitHub URL before submission.

Important evidence note: outputs explicitly identified as expected or illustrative are calculated from the supplied code/rules; they are not claimed to be screenshots or measured runs from the student's computer. Compile and execute every program with the required compiler, capture the actual console output, and replace/add evidence where required.

TABLE OF CONTENTS

1. TASK 1 – Understand the Problem and Identify the Objects (4 Marks)
2. TASK 2 – Use Tokens, Keywords, Identifiers and Constants Correctly (3 Marks)
3. TASK 3 – Choose Data Types and Check Type Compatibility (4 Marks)
4. TASK 4 – Control Implicit Conversions (4 Marks)
5. TASK 5 – Write Correct Expressions with the Right Precedence (4 Marks)
6. TASK 6 – Design the Control Structures (4 Marks)
7. TASK 7 – From Procedural to Object-Oriented (4 Marks)
8. TASK 8 – Read and Print Data Safely (4 Marks)
9. TASK 9 – Implement and Test the Fare Logic (4 Marks)
10. TASK 10 – Compare Expectations with Execution (4 Marks)
11. TASK 11 – Recommend and Defend the Final Design (4 Marks)
12. TASK 12 – Re-evaluate and Adapt the Design (Day-4 Design Challenge) (8 Marks)

Appendix A – Build and execution checklist

Appendix B – GitHub repository structure

TASK 1 – Understand the Problem and Identify the Objects

Marks: 4 | Course Outcome: CO1 | Bloom's Level: K5 – Evaluate

Analysis

The system calculates a fare from the farthest zone travelled and whether the journey starts during peak hours. The card stores value and must retain at least Rs 50 after a fare is deducted. Students and senior citizens receive a concession; the assignment does not specify the concession percentage, so this report treats it as a configurable policy rather than inventing a percentage.

Objects and responsibilities:

- SmartCard: cardId, balancePaise; behaviour: canTravel(fare), deduct(fare), recharge(amount).
- Passenger: passengerId, passengerType; behaviour: identify concession eligibility.
- Journey: sourceZone, destinationZone, hour, day; behaviour: validate journey details.
- Zone: Inner, Middle, Outer (later Airport); represents the fare zone.
- FareTable: base fares and fare rules; behaviour: return the correct base fare.
- FareEngine: combines zone, peak and concession rules to calculate the final fare.

A procedural solution would keep card balance, zone, hour and fare in separate variables and call free functions. An object-oriented solution groups related data and operations, reduces accidental changes and makes the design easier to extend.

OOP principles

Encapsulation: SmartCard keeps balance private and changes it only through recharge() and deduct().

Abstraction: the gate calls calculateFare() without needing to know every arithmetic step.

Inheritance: StudentPassenger and SeniorPassenger may derive from Passenger if they share common passenger data and behaviour.

Polymorphism: a virtual concessionRate() or calculateConcession() can provide different concession rules for passenger types.

Journey invariants: valid zone and hour; fare cannot be negative; a journey is accepted only when balance − fare is at least 5,000 paise; failed journeys do not change the balance; all money calculations use integer paise.

Alternative concept and complexity

With global variables and free functions, the same fare calculation can be implemented procedurally. Both designs take O(1) time per journey and O(1) extra memory per card. OOP does not make an individual calculation faster; its benefit is controlled access and easier maintenance. A fare-rule change in procedural code may require edits at several call sites, while a central FareTable/FareEngine localises the change.

Evidence / assumptions

Assumptions: peak hours are [08:00, 10:00) and [17:00, 20:00); the end time is exclusive. Base fares are Rs 10, Rs 20 and Rs 30. Peak surcharge is 20%. Minimum post-journey balance is Rs 50. The concession amount is unspecified in the source assignment and must be confirmed by the course/fare policy before production use.

TASK 2 – Use Tokens, Keywords, Identifiers and Constants Correctly

Marks: 3 | Course Outcome: CO1 | Bloom's Level: K5 – Evaluate

Explanation

The five token categories are:

- Keywords: int, class, return, if, switch, enum.
- Identifiers: SmartCard, balancePaise, calculateFare.
- Constants: 5000, 20, and named constexpr values.
- Operators: +, -, *, /, %, ==, &&, ||.
- Punctuators: ; , ( ) { } [ ].

Use constexpr for compile-time fare values and percentages. const is appropriate for values that become fixed at run time. #define performs textual substitution and has no normal C++ type or scope. enum class Zone { Inner, Middle, Outer }; is safer than an unscoped enum because enumerators are scoped and do not implicitly convert to int.

Recommended declarations

```
constexpr long long MIN_BALANCE_PAISE = 5000;
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long BASE_FARE_PAISE[] = {1000, 2000, 3000};
enum class Zone { Inner, Middle, Outer };
```

Identifier conventions: use meaningful names; begin with a letter or underscore; do not use spaces; do not use keywords; use consistent camelCase for variables/functions and PascalCase for types. Avoid leading underscores reserved by the implementation in some contexts.

Alternative and complexity

If #define macros were used, the preprocessor would replace names before compilation. Macros have no type checking and can cause precedence surprises unless parenthesised. constexpr and enum class provide type checking and scope. A macro generally occupies no storage; a constexpr object also commonly requires no storage when used as a constant expression. Running time is unchanged: O(1) fare lookup.

TASK 3 – Choose Data Types and Check Type Compatibility

Marks: 4 | Course Outcome: CO1 | Bloom's Level: K5 – Evaluate

Data type selection

Value	Recommended type	Reason
Money in paise	long long	Exact integer arithmetic; large totals
Zone	enum class Zone	Restricted named choices
Hour	int	Values 0–23
Peak flag	bool	True/false
Card identifier	string	May contain letters/leading zeroes
Journey count	long long	Large network totals
Small percentage	int	Whole percentage
Displayed rupees	integer division/remainder	Avoid floating-point money

Money and compatibility

Store Rs 10 as 1,000 paise, Rs 20 as 2,000 paise and Rs 30 as 3,000 paise. Integer paise avoids binary floating-point representation errors. In an expression mixing int and double, the int is converted to double. In a condition, a non-zero arithmetic value converts to true and zero converts to false. Signed/unsigned mixing can convert a negative signed value to a large unsigned value and produce surprising comparisons.

Ranges and scale

The assignment requires <climits> values such as INT_MAX and LLONG_MAX to be checked on the student's compiler. For a conservative upper estimate, 3,000,000 journeys × Rs 30 = Rs 90,000,000 (9 crore rupees), or 9,000,000,000 paise. This exceeds a typical 32-bit signed int maximum of 2,147,483,647 paise, so long long is the safe accumulator. Actual daily collection depends on the mix of fares.

Alternative and complexity

Using double rupees has O(1) addition time and typically 8 bytes per value, like a 64-bit long long on common systems, but it can introduce rounding error. Integer paise is exact for whole-paise amounts. The actual sizeof values are compiler/platform dependent; run sizeof(int), sizeof(long long), sizeof(double) and record the result from the lab machine.

Example C++ check

```
#include <climits>
#include <iostream>
int main() {
    std::cout << "sizeof(int)=" << sizeof(int) << '\\n';
    std::cout << "sizeof(long long)=" << sizeof(long long) << '\\n';
    std::cout << "sizeof(double)=" << sizeof(double) << '\\n';
    std::cout << "INT_MAX=" << INT_MAX << '\\n';
    std::cout << "LLONG_MAX=" << LLONG_MAX << '\\n';
}
```

TASK 4 – Control Implicit Conversions

Marks: 4 | Course Outcome: CO1 | Bloom's Level: K5 – Evaluate

Conversion concepts

Integer division discards the fractional part: 7 / 2 produces 3. Assigning that result to double gives 3.0, not 3.5. Casting before division, static_cast<double>(7) / 2, gives 3.5. A char is promoted to int in arithmetic. A non-zero int assigned to bool becomes true. In a comparison between signed int and unsigned int, the signed operand may be converted to unsigned; therefore -1 can compare as a very large unsigned value.

Predicted and actual output

For the exact program in the assignment, the output is:

3 3.5

C 68

1 true

s >= u

Explanation:

• a / b is integer division before assignment, so c is 3.0 (default stream formatting prints 3).

• static_cast<double>(a) makes the division floating-point, so d is 3.5.

• 'A' + 2 is the character C. ch + 1 promotes ch to int; the character code for C is 67, so the result is 68.

• int value 5 converts to bool true. Default bool output is 1; std::boolalpha prints true.

• In s < u, -1 is converted to unsigned int, becoming a large positive value, so the condition is false and the second string prints.

Safer code and alternative

Use explicit conversions where intent is not obvious and brace initialisation to reject narrowing:

```
int whole{3};
double fraction{static_cast<double>(7) / 2};
bool enabled{5 != 0};
// int invalid{3.7}; // compile-time narrowing error
```

Making conversions explicit generally does not change asymptotic time or memory. It makes intent clearer and lets the compiler reject certain narrowing mistakes at compile time.

TASK 5 – Write Correct Expressions with the Right Precedence

Marks: 4 | Course Outcome: CO1 | Bloom's Level: K5 – Evaluate

Fare expressions

Peak condition:

(hour >= 8 && hour < 10) || (hour >= 17 && hour < 20)

Peak surcharge in paise:

fare += fare * PEAK_SURCHARGE_PERCENT / 100;

Concession should be a separate policy step. Since the assignment does not give the student/senior concession rate, use a named configurable percentage only after confirming the required rate. Avoid hard-coding an invented value.

Predicted and actual output

For the assignment's exact program, the output is:

11 16

9 6 4

6 4

3 1 -3 -1

18

Explanation:

• x + y * 2 evaluates multiplication first: 5 + 6 = 11. Parentheses in (x+y)*2 give 16.

• x++ uses the old x value 5, while ++y increments y from 3 to 4 before use. z = 5 + 4 = 9; x becomes 6 and y is 4.

• x > y is true, so the conditional expression returns 6. x % 4 * 2 is evaluated left to right for equal-precedence multiplicative operators: 6 % 4 × 2 = 4.

• Integer division truncates toward zero: 10/3=3 and -10/3=-3. Remainder follows the dividend sign: -10%3=-1.

• The loop uses i values 0, 3, 6, 9; their sum is 18.

Alternative and complexity

A bool peak[24] lookup table can store whether each hour is peak. Each check is O(1), as is the compound logical expression. The table uses 24 bool elements (commonly 24 bytes, but sizeof(bool) is implementation-defined). It may reduce repeated comparisons, but the logical expression is clearer for this small fixed rule. Avoid expressions such as i = i++, which have problematic sequencing/undefined behaviour depending on the expression and language version; use i++ or ++i as a separate statement.

TASK 6 – Design the Control Structures

Marks: 4 | Course Outcome: CO1 | Bloom's Level: K5 – Evaluate

Pseudocode

```
START
  SET balancePaise to initial stored value
  FOR each trip
    VALIDATE zone and hour
    IF trip is invalid, report invalid trip and CONTINUE
    SET peak according to [08:00,10:00) or [17:00,20:00)
    USE SWITCH on zone to select base fare
    IF peak, add 20 percent using integer arithmetic
    IF balancePaise - farePaise >= 5000
      SUBTRACT farePaise from balancePaise
      PRINT fare and remaining balance
    ELSE
      PRINT journey denied
    END IF
  END FOR
END
```

Completed C++ program

```
#include <iomanip>
#include <iostream>
enum class Zone { Inner, Middle, Outer };

constexpr long long BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;

long long fare(Zone z, bool peak) {
    long long base = 0;
    switch (z) {
        case Zone::Inner:  base = BASE_PAISE[0]; break;
        case Zone::Middle: base = BASE_PAISE[1]; break;
        case Zone::Outer:  base = BASE_PAISE[2]; break;
        default: return -1;
```

```
    }
    if (peak)
        base += base * PEAK_SURCHARGE_PERCENT / 100;
    return base;
}

bool canTravel(long long balance, long long f) {
    return f >= 0 && balance - f >= MIN_BALANCE_PAISE;
}

int main() {
    long long balance = 10000;
    struct Trip { Zone zone; bool peak; };
    Trip trips[] = {{Zone::Inner, false},
                    {Zone::Outer, true},
                    {Zone::Middle, false}};
    for (const Trip& t : trips) {
        long long f = fare(t.zone, t.peak);
        if (canTravel(balance, f)) {
            balance -= f;
            std::cout << "Fare Rs " << f / 100 << '.'
                      << std::setw(2) << std::setfill('0') << f % 100
                      << std::setfill(' ');
        } else {
            std::cout << "Denied (low balance)";
        }
        std::cout << " balance Rs " << balance / 100 << '\\n';
    }
}
```

Output and boundary check

Fare	Rs	10.00	balance	Rs	90
Fare	Rs	36.00	balance	Rs	54

Denied (low balance) balance Rs 54

The source assignment's expected output is reproduced. For boundary testing, hour 10:00 and 20:00 must be off-peak because the end boundary is exclusive; 08:00 and 17:00 are peak. The program above accepts a peak boolean; a production version should calculate it from the validated hour using the expression in Task 5.

Alternative and complexity

An array indexed by a validated zone value gives O(1) fare lookup, as does a switch. The array occupies storage for its elements; a switch may compile to comparisons or a jump table. With a fourth zone, both designs need updating, but a central table can make data-driven extension simpler. Always validate the index before array access.

TASK 7 – From Procedural to Object-Oriented

Marks: 4 | Course Outcome: CO1 | Bloom's Level: K5 – Evaluate

Procedural version

```
#include <iostream>
constexpr long long MIN_BALANCE = 5000;
bool canTravel(long long balance, long long fare) {
    return fare >= 0 && balance - fare >= MIN_BALANCE;
}
bool deduct(long long& balance, long long fare) {
    if (!canTravel(balance, fare)) return false;
    balance -= fare;
    return true;
}
int main() {
    long long balance = 10000;
    long long tripFare = 3000;
    if (deduct(balance, tripFare))
        std::cout << "Accepted, balance paise=" << balance << '\\n';
    else
        std::cout << "Denied\\n";
}
```

Object-oriented version

```
#include <iostream>
class SmartCard {
private:
    long long balancePaise;
    static constexpr long long MIN_BALANCE_PAISE = 5000;
public:
    explicit SmartCard(long long openingBalance)
        : balancePaise(openingBalance) {}
    bool canTravel(long long farePaise) const {
        return farePaise >= 0 &&
               balancePaise - farePaise >= MIN_BALANCE_PAISE;
```

```
    }
    bool deduct(long long farePaise) {
        if (!canTravel(farePaise)) return false;
        balancePaise -= farePaise;
        return true;
    }
    long long balance() const { return balancePaise; }
};
int main() {
    SmartCard card(10000);
    if (card.deduct(3000))
        std::cout << "Accepted, balance paise="
                  << card.balance() << '\\n';
    else
        std::cout << "Denied\\n";
}
```

Comparison and alternative

Both versions use O(1) time for a deduction and O(1) memory for one balance. A public balance member also has the same asymptotic cost, but any code could set it below zero or bypass the Rs 50 rule. A private member with validated methods protects the invariant. When the minimum balance changes, the class centralises the rule in one place.

TASK 8 – Read and Print Data Safely

Marks: 4 | Course Outcome: CO1 | Bloom's Level: K5 – Evaluate

Safe input example

```
#include <iostream>
#include <limits>
int main() {
    int hour;
    std::cout << "Enter hour (0-23): ";
    while (!(std::cin >> hour) || hour < 0 || hour > 23) {
        std::cout << "Invalid hour. Enter 0-23: ";
        if (std::cin.fail()) {
            std::cin.clear();
```

```
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\\n');
        }
    }
    std::cout << "Accepted hour: " << hour << '\\n';
}
```

Formatting

Use <iomanip> to print money consistently. For rupees stored as paise, print amount / 100, a decimal point, and amount % 100 with setw(2) and setfill('0'). For floating-point display values use fixed and setprecision(2), but do not use floating-point values as the authoritative stored money.

Alternative and complexity

With getline(), read one complete line and convert with stoi() inside try/catch. Both approaches take O(L) time to process a line of length L. getline requires a line buffer, so extra space is O(L). Direct cin >> is convenient for simple numeric input; getline plus stoi provides explicit line-level recovery and can reject trailing unwanted characters when carefully validated.

Illustrative output

Enter hour (0-23): abc
Invalid hour. Enter 0-23: 25
Invalid hour. Enter 0-23: 8
Accepted hour: 8

TASK 9 – Implement and Test the Fare Logic

Marks: 4 | Course Outcome: CO1 | Bloom's Level: K5 – Evaluate

Test plan

The following tests cover normal zone fares, peak/off-peak, boundary times, minimum balance, invalid zone, negative recharge and non-numeric hour. The concession percentage is omitted until the official rate is supplied.

Test table

Test	Input / condition	Expected result
Inner off-peak	Inner, 10:00	Rs 10.00
Middle off-peak	Middle, 12:00	Rs 20.00
Outer off-peak	Outer, 12:00	Rs 30.00
Inner peak	Inner, 08:00	Rs 12.00
Middle peak	Middle, 17:00	Rs 24.00

Outer peak	Outer, 19:59	Rs 36.00
Boundary 10:00	Outer, 10:00	Rs 30.00
Boundary 20:00	Outer, 20:00	Rs 30.00
Boundary 08:00	Inner, 08:00	Rs 12.00
Boundary 17:00	Inner, 17:00	Rs 12.00
Exact minimum	Balance Rs 60, fare Rs 10	Accepted; Rs 50 remains
Insufficient	Balance Rs 59.99, fare Rs 10	Denied; balance unchanged
Unknown zone	Zone value outside defined set	Reject input
Negative recharge	Recharge below zero	Reject recharge
Invalid hour text	"abc"	Clear input error and reprompt

Five-journey hand calculation

Assume the following five journeys on one card with an opening balance of Rs 200:

1. Inner off-peak = Rs 10.00
2. Middle peak = Rs 24.00
3. Outer off-peak = Rs 30.00
4. Inner peak = Rs 12.00
5. Middle off-peak = Rs 20.00

Total fare = 10 + 24 + 30 + 12 + 20 = Rs 96.00. Closing balance = Rs 200 − Rs 96 = Rs 104.00. Every journey is accepted because the post-fare balance remains at least Rs 50.

Alternative and complexity

A table-driven test harness stores t input cases and expected results, then loops through them. Execution time is O(t), with O(t) storage for the test table. It is easier to add boundary and regression cases without duplicating test logic.

TASK 10 – Compare Expectations with Execution

Marks: 4 | Course Outcome: CO1 | Bloom's Level: K5 – Evaluate

Collection and integer-range analysis

For a maximum-fare upper-bound estimate of Rs 30 per journey:

20,000 journeys: Rs 600,000 (6 lakh)

500,000 journeys: Rs 15,000,000 (1.5 crore)

3,000,000 journeys: Rs 90,000,000 (9 crore)

In paise these are 60,000,000; 1,500,000,000; and 9,000,000,000. A typical 32-bit signed int cannot hold the last paise total because it exceeds 2,147,483,647. Use long long for the accumulator. These are maximum-fare estimates, not measured collections.

Double versus integer paise

A double can represent many decimal values approximately, so repeated addition may produce a tiny binary rounding difference. Integer paise produces exact totals for fares expressed in whole paise. The actual difference for the assignment's particular fare sequence must be measured by running both accumulators over the same input data; no execution log or journey dataset was included in the assignment document.

Compiler warnings and complexity

Compile with: g++ -std=c++17 -Wall -Wextra -pedantic fare.cpp -o fare

Warnings depend on the student's compiler and exact code. Common warnings include signed/unsigned comparisons, unused variables, implicit narrowing and missing return paths. long long, int and double additions are O(1) time and O(1) storage per accumulator; long long is recommended for range and exact paise representation.

TASK 11 – Recommend and Defend the Final Design

Marks: 4 | Course Outcome: CO1 | Bloom's Level: K5 – Evaluate

Final recommendation

Use enum class for zones; constexpr named fare and percentage constants; long long integer paise for balances and totals; validated input; a FareEngine for fare calculation; and a SmartCard class with private balance and controlled recharge/deduction methods. Use [08:00,10:00) and [17:00,20:00) as the peak windows. Keep concessions as a separately configurable policy once the official student/senior rates are confirmed.

Revision and limitations

A fare revision should be made in the central fare table/constants, followed by recompilation, automated boundary tests and regression tests. The initial switch statement is understandable for three

zones, but a table plus validated zone mapping may be easier when zones are added. The design is O(1) per journey and suitable for high journey counts when totals use long long and each card update is protected. Limitations include the unspecified concession rate, the need for persistent/transaction-safe card storage, concurrency at gates, refunds, audit logs and secure card authentication.

Alternative concept

Reading z fare values from a configuration file adds O(z) start-up time and O(z) storage. It avoids recompilation for fare-only changes but requires strict validation, access control, versioning and rollback. Compiled constants require rebuild/redeployment but are simpler to validate and package. A controlled configuration file is preferable when fare changes are frequent and operational governance is strong.

TASK 12 – Re-evaluate and Adapt the Design (Day-4 Design Challenge)

Marks: 8 | Course Outcome: CO1 | Bloom's Level: K5 – Evaluate

A. Impact analysis

The new Airport zone adds a fourth enum value and Rs 60 base fare (6,000 paise). Fare tables, zone validation, switch logic, test cases, display/reporting and any zone-count assumptions must change. Weekend discount introduces a day-of-week input and a discount rule. The 10% weekend discount must not combine with the peak surcharge. A monthly pass requires a new Pass object with validity dates and two selected zones, plus SmartCard/pass integration.

B. Evaluate original design

A switch can support Airport by adding a case, but repeated switch statements across the program increase maintenance risk. A central fare table is easier to extend. Worked examples:

- Airport weekday off-peak: Rs 60.00.
- Airport weekday peak: Rs 72.00 (20% surcharge).
- Airport weekend: Rs 54.00 (10% discount, no peak surcharge even if the time is in a peak window).
- Inner weekend: Rs 9.00.

The weekend rule takes precedence over peak surcharge, as specified.

C. Revised design code

```
#include <iostream>
#include <array>
#include <string>
enum class Zone { Inner, Middle, Outer, Airport };
enum class DayType { Weekday, Weekend };
constexpr std::array<long long, 4> FARES = {1000, 2000, 3000, 6000};
```

```
long long calculateFare(Zone zone, DayType day, bool peak) {
    auto index = static_cast<int>(zone);
    if (index < 0 || index >= static_cast<int>(FARES.size()))
        return -1;
    long long amount = FARES[static_cast<std::size_t>(index)];
    if (day == DayType::Weekend)
        amount = amount * 90 / 100;       // no peak surcharge on weekends
    else if (peak)
        amount = amount * 120 / 100;
    return amount;
}
class MonthlyPass {
    int zoneA, zoneB;
    bool active;
public:
    MonthlyPass(int a, int b) : zoneA(a), zoneB(b), active(true) {}
    bool covers(Zone z) const {
        int value = static_cast<int>(z);
        return active && (value == zoneA || value == zoneB);
    }
};
int main() {
    std::cout << calculateFare(Zone::Airport, DayType::Weekday, true) / 100.0 << '\\n';
    std::cout << calculateFare(Zone::Airport, DayType::Weekend, true) / 100.0 << '\\n';
}
```

Illustrative output

72

54

The displayed numeric output is in rupees. A production implementation should format paise to exactly two decimal places and validate the zone enum before converting it to an array index.

D. Evidence-based defence

Decision: MODIFY the original design. Keep the SmartCard encapsulation and integer-paise arithmetic, but replace scattered switch-based fare selection with one validated fare table and a separate

mutually exclusive weekend/weekday pricing policy. Add a MonthlyPass class. This retains proven invariants while reducing the cost of adding future zones. Required verification includes all four zones, weekday/weekend, peak/off-peak boundaries, pass coverage, expiry and insufficient-balance cases. The assignment document does not provide measured execution results; the examples here are expected/illustrative and must be confirmed by running the programs.

APPENDIX A – BUILD AND EXECUTION CHECKLIST

Recommended compiler command (as required by the assignment):

```
g++ -std=c++17 -Wall -Wextra -pedantic filename.cpp -o program
```

Run the executable, test valid and invalid inputs, save the terminal output as a screenshot, and retain the source and input files in the corresponding task folder. Record the compiler name/version and exact command in README.md.

Before final submission, verify: (1) all 12 tasks are present; (2) every source file compiles; (3) sample outputs match actual runs; (4) screenshots are genuine; (5) GitHub repository is accessible to the faculty; (6) one or more commits exist for each task.

APPENDIX B – SUGGESTED GITHUB REPOSITORY STRUCTURE

```
DSA01-<RegisterNumber>-Assignment1/
 README.md
 Task01/
  analysis.md
  output.png
 Task02/
  constants.cpp
  output.png
 Task03/
  datatype_check.cpp
  output.png
 ...
 Task12/
  revised_fare.cpp
  output.png
 evidence/
  test-results/
```

REFERENCES

1. Object Oriented Programming with C++, Assignment 1 – CO1, supplied course document (13 pages).

2. ISO C++ language facilities referenced in the assignment: standard integer types, enum class, constexpr, static_cast, iostream and iomanip.

3. Compiler documentation for the C++17 compiler used in the lab; enter the exact compiler/version in the final README.